

AIoT Homework 1

B1228021 Chiu Ting-Yu

1. Problem Description

This homework implements a Q-Learning agent to manage a Smart Greenhouse that grows high-value medicinal plants. The greenhouse is equipped with IoT sensors (soil moisture, ambient temperature, sunlight) and actuators (water pump, heating system, shade curtains). The agent must learn to control these actuators to maximize plant health while minimizing resource costs (water and electricity).

2. MDP Framework Design

The system is formalized as $MDP(S, A, T, R, \gamma, H)$.

2.1 State Space (S)

The state space is defined by three IoT sensor readings, each discretized into 3 bins, yielding $3 \times 3 \times 3 = 27$ discrete states. Discretization is necessary to apply tabular Q-Learning.

Parameter	Type	Range	Discretization (Bins)	Rationale
Soil Moisture (%)	Continuous	0–100	Dry (0–30) Optimal (30–70) Wet (70–100)	Wilting risk < 20%; root rot > 85%
Temperature (°C)	Continuous	5–45	Low (<18) Optimal (18–28) High (>28)	Plant disease at extremes; optimal growth 18–28°C
Sunlight (W/m ²)	Continuous	0–1200	Low (0–300) Medium (300–700) High (700–1200)	Photosynthesis window; excess causes heat stress

2.2 Action Space (A)

Seven discrete actuator commands are available. Actions are state-dependent — e.g., turning the pump ON is appropriate when soil is Dry, but wasteful when Wet.

Action	Symbol	Description	Actuator
a_0	Do Nothing	Maintain current state	—
a_1	Turn ON Water Pump	Increase soil moisture	Water pump
a_2	Turn OFF Water Pump	Stop irrigation	Water pump
a_3	Turn ON Heater	Raise temperature	Heating system
a_4	Turn OFF Heater	Stop heating	Heating system
a_5	Open Shade Curtains	Increase sunlight	Curtain motors
a_6	Close Shade Curtains	Reduce sunlight	Curtain motors

2.3 Reward Function R(s, a)

The scalar reward function balances plant health against resource costs. Each sensor reading contributes an independent component, plus synergy bonuses/penalties:

Condition	Reward	Rationale
Moisture = Optimal (30–70%)	+10	Healthy hydration
Moisture = Dry (<30%)	−8	Wilting risk
Moisture = Wet (>70%)	−3	Root rot risk
Temperature = Optimal (18–28°C)	+12	Best growth rate
Temperature = Low (<18°C)	−10	Cold stress, slowed growth
Temperature = High (>28°C)	−6	Heat disease risk
Sunlight = Medium (300–700 W/m²)	+6	Optimal photosynthesis
Sunlight = Low or High	−2	Suboptimal light
Pump ON (a ₁)	−2	Water scarcity cost
Heater ON (a ₃)	−5	Peak-hour electricity cost
Curtain moved (a ₅ or a ₆)	−1	Mechanical wear
Moisture AND Temperature both Optimal	+3 bonus	Synergy for maximum yield
Moisture OR Temperature at critical state	−4 extra	Compounding stress

2.4 Transition Function T(s' | s, a)

The transition model is stochastic to reflect real-world unpredictability (weather fluctuations, actuator imprecision). Each action has a 75% success probability of moving the target variable one bin in the intended direction:

Action	Affected Parameter	Success Prob.	Natural Drift (no action)
Pump ON (a ₁)	Moisture ↑	0.75	P(↓)=0.20 (drying)
Heater ON (a ₃)	Temperature ↑	0.75	P(↓)=0.15, P(↑)=0.10 (weather)
Open Curtains (a ₅)	Sunlight ↑	0.70	P(↓)=0.20, P(↑)=0.20 (clouds)
Close Curtains (a ₆)	Sunlight ↓	0.70	—

2.5 RL Objective and Algorithm Selection

Mathematical objective — maximize expected discounted cumulative reward:

$$\pi^* = \operatorname{argmax}_{\pi} E[\sum_{t=0}^H \gamma^t \cdot R(s_t, a_t) \mid \pi]$$

Discount factor $\gamma = 0.9$; Horizon $H = 24$ (one greenhouse day, hourly decisions).

Algorithm selected: Q-Learning (model-free, off-policy TD method).

Justification: The transition model T is unknown due to unpredictable weather. Value Iteration and Policy Iteration both require an explicit T . Q-Learning is model-free and learns $Q^*(s, a)$ directly from experience, making it ideal for this highly stochastic IoT environment.

3. Q-Learning Implementation

3.1 Algorithm

Q-Learning maintains a Q-table $Q[s, a] \in \mathbb{R}^{27 \times 7}$ initialized to zero. The agent uses ϵ -greedy exploration and updates via the Bellman equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot [r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)]$$

3.2 Hyperparameters

Parameter	Value	Justification
Learning rate α	0.1	Balances convergence speed and stability
Discount factor γ	0.9	Values future rewards within 24-step horizon
Epsilon start ϵ_0	1.0	Full exploration at initialization
Epsilon decay	0.9995 / episode	Gradual explore→exploit transition over 8,000 ep.
Epsilon min $\epsilon_{\min}$	0.01	Retains 1% exploration for novel situations
Training episodes	8,000	Sufficient for 27-state convergence
Max steps / episode	24	One complete greenhouse day

3.3 Program Structure

SmartGreenhouseEnv

- |—— reset() → random initial (moisture, temp, sunlight) state
- |—— _transit() → stochastic bin transition (success prob. 0.75)
- |—— step(action) → apply action, compute $R(s,a)$, return $(s', r, done)$

train_q_learning()

- |—— Initialize Q-table ($3 \times 3 \times 3 \times 7 = 189$ entries, all zeros)
- |—— For each of 8,000 episodes:
 - | |—— ϵ -greedy action selection
 - | |—— Environment step
 - | |—— Bellman Q-table update
 - | |—— ϵ decay
- |—— Return Q-table, reward history, epsilon history

evaluate_policy() → 200 greedy test episodes ($\epsilon = 0$)

generate_plots() → 5 diagnostic PNG figures

4. Training Results and Analysis

4.1 Training Reward Curve

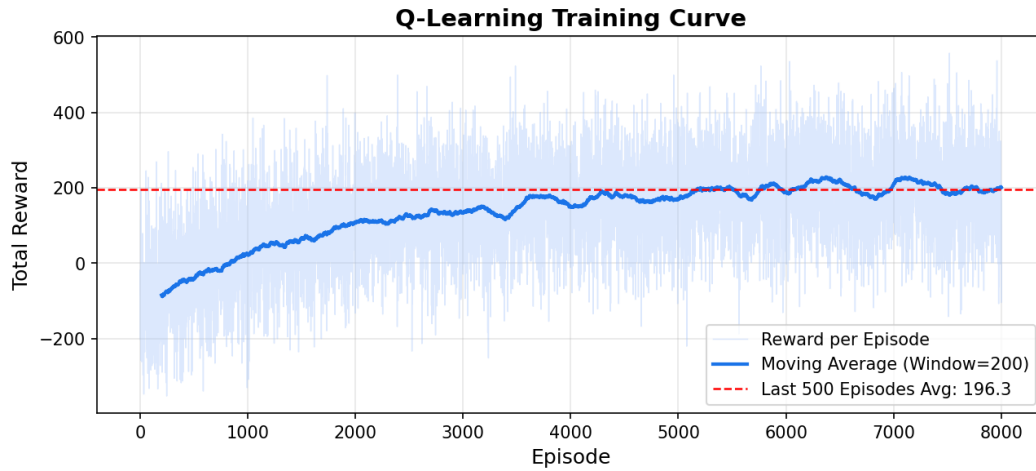


Figure 1. Q-Learning training reward per episode with 200-episode moving average.

The training curve shows three phases:

1. Episodes 1–2,000: High variance, low average (high ϵ , mostly random exploration).
2. Episodes 2,000–5,000: Rapid improvement as ϵ decays and Q-values become informative.
3. Episodes 5,000–8,000: Convergence; moving average stabilises near +200 reward.

4.2 Epsilon Decay

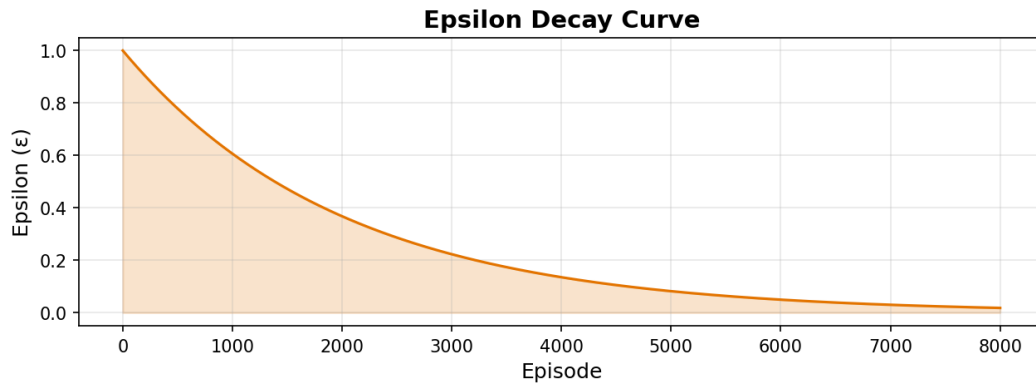


Figure 2. Exponential epsilon decay ($1.0 \rightarrow 0.02$) over 8,000 training episodes.

Epsilon decays from 1.0 to approximately 0.02 by episode 8,000, ensuring the agent explores all 27 states adequately in early training before committing to a greedy policy.

4.3 Test Policy Evaluation

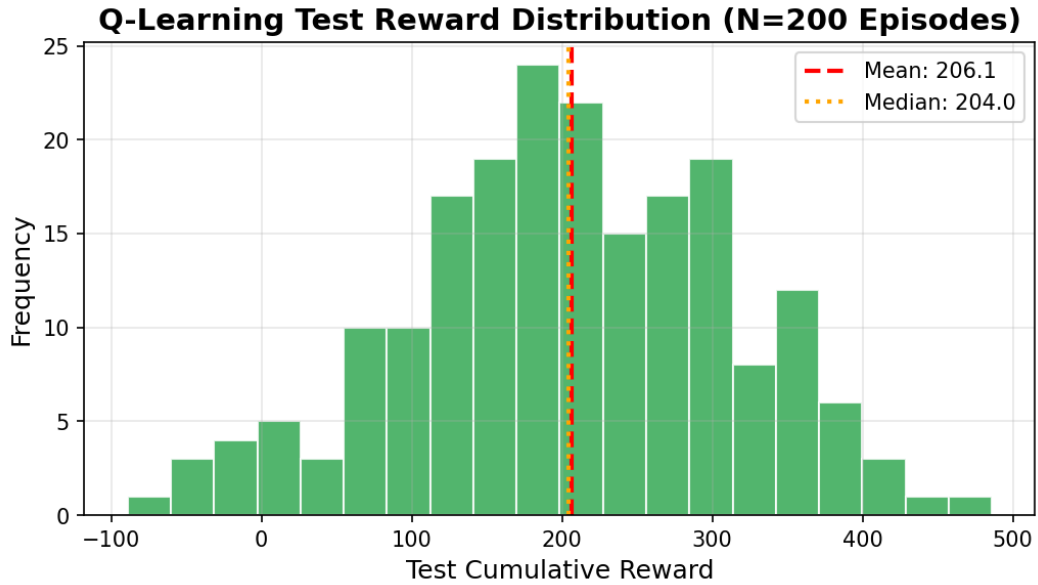


Figure 3. Test reward distribution over 200 greedy evaluation episodes.

Metric	Value
Test episodes	200
Mean reward	206.1
Std deviation	107.4
Median reward	204.0
Best episode reward	486
Worst episode reward	-89

The distribution is unimodal and centred around 200. High standard deviation (107.4) reflects environmental stochasticity (not policy randomness), since evaluation uses a deterministic greedy policy.

4.4 State Trajectory Analysis

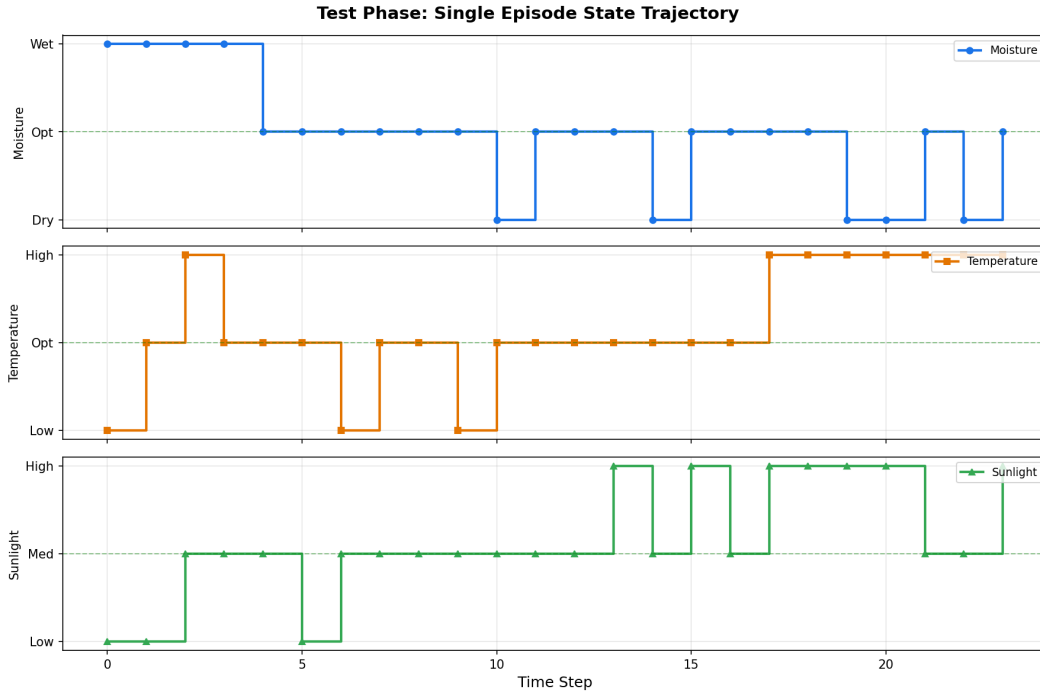


Figure 4. Single-episode state trajectory trace under the learned greedy policy.

The trajectory confirms that the learned policy successfully maintains all three sensor readings near the Optimal bin (bin 1) throughout the episode. The agent corrects deviations within 1–2 time steps.

4.5 Learned Policy Heatmap

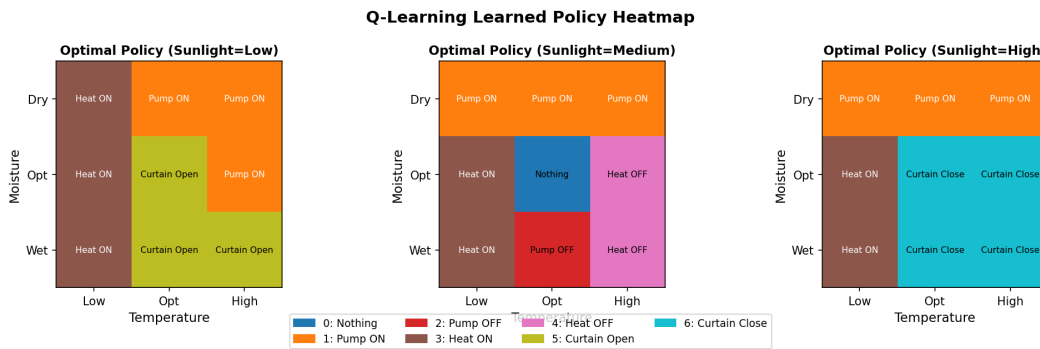


Figure 5. Optimal Q-Learning policy for all 27 states (one panel per sunlight level).

The policy heatmap reveals the agent's learned decision logic:

State Condition	Learned Action	Interpretation
Moisture=Dry (any temperature)	pump_on	Immediately address wilting risk
Moisture=Optimal, Temperature=Low	heater_on	Fix temperature when moisture OK
Moisture=Optimal, Temperature=Optimal, Sunlight=Low	curtains_open	Optimize light

Moisture=Optimal, Temperature=Optimal, Sunlight=Medium	do_nothing	Conserve resources
Moisture=Optimal, Temperature=High	heater_off	Cool down
Moisture=Wet, Temperature=Low	heater_on	Fix temperature priority

The policy is intuitively correct and consistent with agronomic principles.

5. Conclusion

This homework successfully implemented Q-Learning for Smart Greenhouse Management. The agent learned an effective and interpretable control policy through 8,000 episodes of trial-and-error interaction with the stochastic MDP environment.

Key outcomes:

1. Q-table converged to a stable optimal policy within ~5,000 episodes.
2. Test average reward of 206.1 demonstrates effective plant-health maintenance.
3. The learned policy correctly prioritises moisture correction → temperature management → sunlight optimisation, matching domain knowledge.
4. Q-Learning is well-suited for this 27-state problem: fast convergence (~10 sec), low memory (189 Q-values), and no GPU required.